FERRO D2

v4.0 — LlamaIndex Edition

Documentación Maestra Completa

Arquitectura de Referencia para Sistemas de IA Empresarial Multi-Tenant con Integraciones Bidireccionales y Transformación Documental

Estado	Production Ready
Edición	Febrero 2026
Versión	4.0 (Consolidada + Integraciones)
Autoría	Zeron Architecture Team
Clasificación	Confidencial — Uso Interno

© 2026 Zeron Architecture Team / Antigravity

Índice de Contenidos

Índice de Contenidos 2
Parte I − Visión y Whitepaper 4
1. Resumen Ejecutivo 4
2. El Problema que Resuelve 4
2.1 El Techo de Cristal del GIL 4
2.2 La Trampa del Payload Vectorial 5
2.3 Integraciones sin Modelar 5
2.4 Transformación Bidireccional Ausente 5
3. La Solución FERRO D2 5
Parte II − El Stack FERRO: Cada Componente 6
4. F − FastAPI: Orquestador Asíncrono y API Gateway 6
5. E − Engine (LlamaIndex): Framework Data-Centric 6
6. R − React: Frontend Dopamine 7
7. R − Rust (PyO3): Motor de Cálculo Nativo 7
8. O − Ops/DataOps: Infraestructura Inmutable 7
Parte III − Estrategia de Datos D2 (Persistencia Políglota) 8
9. Redis − Capa de Velocidad (Dopamine Layer) 8
10. PostgreSQL − Capa de Verdad (Relational Core) 8
11. MongoDB − Capa de Memoria y Logs 9
12. Qdrant / LanceDB − Capa Semántica 9
12.1 Estrategia de Escalado Multi-Tenant en Qdrant 9
Parte IV − Arquitectura de Flujos 11
13. Diagrama de Arquitectura (Alto Nivel) 11
14. Pipeline de Ingestión (Write Path) 11
15. Pipeline RAG (Read Path) 11
Parte V − Pipeline de Transformación Documental (Bidireccional) 13
16. Visión General 13
17. Flujo de Estados del Job 13
18. Endpoints del Pipeline de Transformación 14
19. Contratos de Entrada y Salida 14
19.1 TransformRequest (entrada) 14
19.2 TransformResponse (salida de status) 14

20. Capa de Rendering de Documentos ... 15
Parte VI − Capa de Integraciones ... 16
21. Adapter Pattern para Conectores ... 16
22. Registro de Integraciones en PostgreSQL ... 16
23. Colas de Trabajo con Prioridad por Tenant ... 17
24. Circuit Breakers por Integración ... 17
Parte VII − Metering y Costos por Tenant ... 18
25. Por Qué es Imprescindible ... 18
26. Modelo de Datos ... 18
27. Qué se Mide ... 18
28. Quotas y Límites ... 19
Parte VIII − Seguridad End-to-End ... 20
29. Principios de Seguridad ... 20
30. Modelo de Amenazas ... 20
31. RBAC + ABAC + RLS ... 21
32. Seguridad del Pipeline de Transformación ... 21
33. Seguridad del Bridge Rust (PyO3) ... 21
34. Seguridad Operativa ... 21
Parte IX − Observabilidad y Operaciones ... 22
35. Observabilidad ... 22
36. Runbooks Operativos ... 22
37. Healthchecks Obligatorios ... 22
Parte X − Guía de Implementación ... 23
38. Estructura del Monorepo ... 23
39. Motor Rust: Cargo.toml y lib.rs ... 23
40. Bridge LlamaIndex + Rust ... 24
41. DDL Base: Multi-Tenant + RLS + Integraciones + Metering ... 24
42. Docker Compose (Esqueleto con Broker) ... 26
Parte XI − Degradación Elegante y Fallbacks ... 28
Parte XII − Las Reglas de Hierro (Invariantes) ... 29
Parte XIII − Objetivos No Funcionales (NFRs) ... 29
Parte XIV − Checklist Go-Live ... 31
Anexo A − AI System Skills (Prompt Operativo) ... 33

Parte I — Visión y Whitepaper

1. Resumen Ejecutivo

FERRO D2 v4 es una arquitectura de referencia híbrida de alto rendimiento diseñada para construir sistemas de IA empresarial multi-tenant con integraciones bidireccionales. Combina procesamiento cognitivo (RAG + agentes) con transformación documental, permitiéndole recibir datos de sistemas externos, procesarlos con IA, y devolver documentos estructurados.

El sistema se articula en torno a tres pipelines principales: el Pipeline RAG (consulta conversacional con contexto), el Pipeline de Ingestión (procesamiento y vectorización de documentos) y el Pipeline de Transformación Documental (recepción de datos, resumen con IA y devolución de documentos formateados).

Métrica Clave	Objetivo	Mecanismo
Latencia percibida	Respuesta inmediata ("Dopamine UX")	Redis cache < 5ms + Optimistic UI + Streaming
Integridad y gobernanza	ACID + auditoría por tenant	PostgreSQL con RLS + RBAC/ABAC determinista
Throughput de ingestión	Procesamiento masivo paralelo	Rust (PyO3 + Rayon) sin GIL
Integraciones bidireccionales	Recibir JSON → devolver documento	Cola de jobs + LLM + rendering de documentos
Aislamiento multi-tenant	Cero fuga entre tenants	RLS + namespaces + filtros en cada capa
Metering por tenant	Costo y consumo trazable	Contadores de embeddings, LLM calls, storage

Principio operativo
"La velocidad no vale nada si filtra datos." PostgreSQL y sus políticas RLS mandan sobre cualquier retrieval semántico. Si Postgres dice NO, la consulta semántica no ocurre.

2. El Problema que Resuelve

2.1 El Techo de Cristal del GIL

En sistemas RAG multi-tenant, múltiples tenants ingieren documentos concurrentemente. El Global Interpreter Lock (GIL) de Python impide el paralelismo real en operaciones CPU-bound (chunking, tokenización, normalización), creando cuellos de botella críticos cuando 20+ tenants suben documentos simultáneamente.

2.2 La Trampa del Payload Vectorial

Almacenar metadatos de negocio (permisos, estados, fechas) dentro del JSON del vector store provoca degradación de índices HNSW por pre-filtrado excesivo, pérdida de garantías ACID y datos sin gobernanza. En multi-tenant, esto amplifica el riesgo de fuga cross-tenant.

2.3 Integraciones sin Modelar

La mayoría de los sistemas RAG tratan la ingestión como si los datos llegaran mágicamente. En la realidad, el 60% de los bugs viven entre la fuente externa y tu pipeline. Sin un patrón explícito de conectores, colas de trabajo por tenant y circuit breakers por integración, un conector que falla puede degradar el servicio de todos los tenants.

2.4 Transformación Bidireccional Ausente

Los sistemas RAG típicos solo responden preguntas. FERRO D2 necesita además recibir datos estructurados (JSON), procesarlos con IA (resumen, enriquecimiento) y devolver documentos formateados al usuario o sistema que los pidió. Esto requiere un pipeline de transformación documental con jobs asíncronos, estados persistentes y entrega confiable.

3. La Solución FERRO D2

Se implementa una arquitectura de seis capas: un puente FFI (Rust/PyO3) para cálculo paralelo, persistencia políglota con cuatro bases de datos especializadas, una capa de integraciones con adapter pattern y message broker, y un pipeline de transformación documental completo con jobs asíncronos.

Parte II — El Stack FERRO: Cada Componente

El acrónimo FERRO define cinco capas tecnológicas. La estrategia D2 agrega la capa de datos políglota. Y las integraciones bidireccionales añaden una capa de conectores y transformación.

Capa	Tecnología	Rol	Naturaleza
F	FastAPI	Gateway + Orquestador	I/O, validación, authN/Z, webhooks
E	Engine (LlamaIndex)	Pipeline data-centric	Ingestión, indexación, retrieval, síntesis
R	React	Frontend Dopamine	UI optimista, streaming, estados de job
R	Rust (PyO3)	Motor de cálculo nativo	CPU-bound puro, paralelo, sin I/O
O	Ops / DataOps	Infraestructura inmutable	Docker, CI/CD, observabilidad, backups
D2	Redis/PG/Mongo/Qdrant	Persistencia políglota	Cada dato donde rinde mejor
+	Message Broker	Cola de trabajos	Jobs asíncronos, prioridad por tenant
+	Doc Renderer	Generación de documentos	Salida formateada (JSON/PDF/DOCX)

4. F — FastAPI: Orquestador Asíncrono y API Gateway

Qué hace: Recibe requests HTTP/webhooks, valida payloads con Pydantic, aplica autenticación y autorización, enruta a los pipelines correspondientes (RAG, ingestión o transformación) y coordina toda la ejecución de I/O.

Por qué se elige: Combina performance en I/O (Starlette/Uvicorn), tipado estricto con Pydantic, OpenAPI automático para documentar las APIs de integración, y soporte nativo para webhooks, background tasks y WebSockets/streaming.

> **Concepto clave: Airlock**
> Ningún dato externo llega al motor de Rust ni a las bases de datos sin pasar primero por validación estricta de Pydantic. Esto previene desbordamientos de memoria, inyecciones y datos malformados de integraciones externas.

Endpoints mínimos: /v1/auth/login, /v1/docs/ingest, /v1/chat/query, /v1/transform/submit, /v1/transform/status/{job_id}, /v1/transform/result/{job_id}, /v1/index/rebuild, /v1/integrations/{id}/webhook, /healthz, /readyz, /version.

5. E — Engine (LlamaIndex): Framework Data-Centric

Qué hace: Define el pipeline completo: Document → Nodes → Index → Retriever → Response Synthesizer. Permite inyectar Rust como transformación determinista y testear cada etapa de forma aislada.

Por qué LlamaIndex: Prioriza el pipeline de datos como ciudadano de primera clase. Permite modularidad (cambiar retriever, embeddings o LLM sin reescribir la app) y reduce acoplamiento.

> **Precaución**
> LlamaIndex cambia de API con frecuencia (la migración v0.9 a v0.10 fue disruptiva). Se recomienda mantener una capa de abstracción propia (engine/rust_bridge.py y engine/llamaindex_pipeline.py) entre LlamaIndex y tu código para aislar el impacto de actualizaciones del framework.

6. R — React: Frontend Dopamine

Qué hace: UI moderna con Optimistic UI, streaming de tokens, y manejo explícito de estados de jobs asíncronos (para transformación documental: enviado → procesando → listo → descargar).

Patrones recomendados: TanStack Query para cache y reintentos, estados explícitos de progreso para uploads e integraciones, polling o WebSocket para estado de jobs de transformación, y manejo explícito de degradación.

Componentes sugeridos: ChatPanel, DocUploader, DocLibrary, TransformJobTracker, IntegrationsDashboard, AdminIndexPage y Settings/Roles.

7. R — Rust (PyO3): Motor de Cálculo Nativo

Qué hace: Ejecuta chunking con offsets, tokenización, normalización unicode/whitespace, hashing (doc_hash y chunk_hash), deduplicación y preprocesamiento en paralelo real con Rayon.

Qué NO hace: Rust nunca realiza I/O. Sin red, sin DB, sin filesystem. Funciones puras con límites defensivos: max_input_bytes, max_chunks, max_chunk_size, overlap_range, retorno controlado (Result).

Justificación multi-tenant: Con 20+ tenants ingiriendo documentos concurrentemente desde múltiples integraciones, el GIL se convierte en un bottleneck medible. Rust escala linealmente con cores disponibles, procesando documentos de minutos a segundos (hasta 100x más rápido).

8. O — Ops/DataOps: Infraestructura Inmutable

Docker multi-stage para builds reproducibles, CI/CD con lint + tests + security scanning, gestión de secretos fuera del repositorio (vault/secret manager), observabilidad integrada (Prometheus/Grafana + logs estructurados + OpenTelemetry), backups cifrados con restore drills periódicos, y runbooks documentados para cada operación crítica.

Parte III — Estrategia de Datos D2 (Persistencia Políglota)

La decisión más crítica en FERRO D2 es dónde guardar o leer cada bit de información. El D2 Router clasifica los datos según su naturaleza y los envía a la base que mejor los sirve.

Capa	Tecnología	Qué guarda	Regla principal
Velocidad	Redis	Cache, sesiones, rate limit, locks, job status	"First Hit Cache" — verificar antes de procesar
Verdad	PostgreSQL	Tenants, usuarios, roles, permisos, documentos, jobs, integraciones, metering, auditoría	ACID obligatorio — fuente de verdad
Memoria	MongoDB	Chat history, trazas LLM, raw docs, metadata de integraciones	Escritura rápida, esquema flexible, TTL
Semántica	Qdrant / LanceDB	Embeddings + payload mínimo	Solo vectores y referencias compactas

9. Redis — Capa de Velocidad (Dopamine Layer)

Redis habilita la "Dopamine Rule": toda lectura verifica primero el cache. Reduce latencia a < 5ms y disminuye costos de LLM y DB significativamente.

Patrones: Cache-aside (GET → compute → SETEX), locks de idempotencia (SET NX EX), rate limiting por tenant/usuario (token bucket), session store con expiración, y status de jobs de transformación para polling rápido.

Claves: cache:{tenant}:{hash(q+ctx)}, sess:{tenant}:{user}:{session}, rl:{tenant}:{user}:{min}, lock:ingest:{doc_hash}, job:{job_id}:status, idxver:{tenant}.

> Seguridad: Namespaces por tenant con hash de policy en la key. Nunca almacenar datos sensibles sin cifrado. Expiración obligatoria en todas las claves.

10. PostgreSQL — Capa de Verdad (Relational Core)

Fuente de verdad absoluta. Almacena tenants, usuarios, roles, permisos, políticas ABAC, documentos (estado), facturación, registro de integraciones, jobs de transformación, metering de consumo y auditoría completa.

RLS (Row Level Security): Enforcement server-side que garantiza que cada query devuelva solo filas del tenant permitido. Aunque la app tenga un bug, la DB limita lo que se puede leer.

ABAC determinista: Políticas evaluadas contra atributos del sujeto, recurso, acción y contexto. Cada evaluación produce policy_id, inputs, resultado y motivo para auditoría.

Nuevo: Registro de integraciones: Tabla integrations con: tenant_id, connector_type, credentials_ref (referencia al vault, nunca plaintext), last_sync_at, health_status, config JSONB, y rate_limit_config.

Nuevo: Metering por tenant: Tabla usage_metering con contadores de embeddings generados, llamadas LLM, tokens consumidos, storage en Qdrant/Mongo, y documentos procesados. Base para facturación y quotas.

11. MongoDB — Capa de Memoria y Logs

Almacena historial conversacional, trazas de prompts/responses, metadata no estructurada de integraciones, y los inputs/outputs crudos de jobs de transformación (para reprocesamiento y auditoría).

Patrón bucket: Agrupar mensajes por sesión en bloques de 50–200 para evitar documentos enormes.

Colecciones separadas: llm_traces (TTL), chat_sessions (retención configurable), audit_events (append-only), transform_payloads (input JSON original + output generado), integration_raw_data (metadata heterogénea de conectores).

TTL indexes y scrub de secretos obligatorio. Write concern bajo para trazas (fire and forget), alto para eventos críticos.

12. Qdrant / LanceDB — Capa Semántica

Almacena únicamente embeddings y payload mínimo: tenant_id, doc_id, chunk_id, timestamp y opcionalmente doc_hash. Nunca lógica mutable ni campos de negocio.

> **Índice Derivado**
> Qdrant NO es la verdad; se reconstruye completamente desde PostgreSQL/MongoDB. Si se corrompe o pierde, se regenera siguiendo el pipeline de ingestión.

12.1 Estrategia de Escalado Multi-Tenant en Qdrant

Con muchos tenants, hay dos estrategias posibles y un punto de inflexión crítico que debe monitorearse:

Estrategia	Cómo funciona	Ventaja	Punto de inflexión
Colección compartida	Una colección con filtro por tenant_id	Simple, menos overhead operativo	Degradación cuando el índice crece y el filtrado se vuelve costoso (~5M+ vectores)
Colección por tenant	Una colección dedicada por tenant	Aislamiento fuerte, rendimiento predecible	Overhead operativo con muchos tenants (~100+), más RAM por colección

Estrategia	Cómo funciona	Ventaja	Punto de inflexión
Híbrido	Compartida para tenants pequeños, dedicada para grandes	Balance óptimo	Requiere lógica de routing y migración entre estrategias

Recomendación: comenzar con colección compartida + cuantización int8 (reduce RAM 4x). Monitorear latencia de búsqueda. Migrar a colección dedicada para tenants que superen un umbral de volumen (configurable).

Parte IV — Arquitectura de Flujos

13. Diagrama de Arquitectura (Alto Nivel)

```
[React UI]  ──  HTTPS  ──>  [Nginx / Traefik]  ──  metrics  ──>  [Prometheus/Grafana]
                                  |
                                  v
                          [FastAPI API Gateway]
                           |     |     |     |      |        |
                         Redis  PG   Mongo  Qdrant  Rust   Broker
                         cache  RLS  logs   vectors PyO3   jobs
                         sess   auth traces payload chunk  queues
                         rate   jobs raw    min     hash   async
                         locks  meter                      transform

[Sistemas Externos]  ──  webhook/API  ──>  [FastAPI]  ──>  [Broker]  ──>  [Worker]
```

14. Pipeline de Ingestión (Write Path)

Secuencia estricta donde cada paso existe por una razón específica:

1. **Validación (FastAPI + Pydantic):** Evita inputs maliciosos, fija límites de tamaño, tipo y encoding. Para integraciones, valida también el schema del conector.
2. **Lock Redis (SETNX doc_hash):** Idempotencia y ahorro de CPU ante reintentos o duplicados de integraciones.
3. **Registro PostgreSQL (status=PROCESSING):** Fuente de verdad de estados; permite tracking del job y metering.
4. **Transformación Rust (PyO3):** Chunking, tokenización y hashing paralelo sin GIL.
5. **Embeddings (batch):** Generación de vectores con costo controlado. Metering: contabilizar tokens consumidos.
6. **Upsert Qdrant:** Índice semántico derivado con payload mínimo.
7. **Persistencia MongoDB:** Texto crudo, chunks y logs del job (async).
8. **Confirmación PostgreSQL (status=AVAILABLE):** Habilita queries. Incrementa contadores de metering.
9. **Cache invalidation (Redis):** Coherencia de respuestas cacheadas para el tenant.

15. Pipeline RAG (Read Path)

Prioriza velocidad (cache) y seguridad (autorización antes de retrieval):

1. **Cache-aside (Redis):** Si existe respuesta cacheada, retorna en < 5ms.
2. **Autorización (PostgreSQL):** RBAC/ABAC obtiene lista de doc_ids permitidos para el usuario/tenant.
3. **Retrieval (Qdrant):** Búsqueda semántica con filtros simples (tenant_id + doc_ids autorizados).
4. **Memoria de sesión (MongoDB):** Últimos N mensajes para coherencia conversacional.
5. **Generación (LLM):** Respuesta final con streaming de tokens. Metering: contabilizar llamada.
6. **Persistencia (MongoDB + Redis):** Trazas async + cache del resultado si es cacheable.

Parte V — Pipeline de Transformación Documental (Bidireccional)

Nuevo en esta versión
Este pipeline es el que habilita integraciones bidireccionales: recibir datos de sistemas externos, procesarlos con IA, y devolver documentos estructurados al usuario o al sistema que los solicitó.

16. Visión General

A diferencia del pipeline RAG (pregunta → contexto → respuesta), el pipeline de transformación opera sobre datos completos: un sistema externo envía un JSON con información estructurada, FERRO lo procesa (parseo, resumen con LLM, enriquecimiento), y devuelve un documento formateado (JSON resumido, PDF, DOCX) al solicitante.

Este flujo es necesariamente asíncrono: el LLM puede tardar 5–30 segundos, y el sistema externo probablemente tiene timeouts de 30 segundos o menos. Por eso se implementa con un modelo de jobs: recibir, encolar, procesar, notificar.

17. Flujo de Estados del Job

```
RECEIVED  —>  QUEUED  —>  PROCESSING  —>  SUMMARIZING  —>  RENDERING  —>  COMPLETED
    |             |             |              |              |             |
    v             v             v              v              v             v
  FAILED        FAILED        FAILED         FAILED         FAILED      (Entrega)
                                                                          webhook
                                                                          o polling
```

Estado	Qué ocurre	Dónde se persiste
RECEIVED	JSON recibido y validado por Pydantic	PostgreSQL (transform_jobs) + MongoDB (payload crudo)
QUEUED	Job encolado en el message broker con prioridad por tenant	Redis (job:{id}:status) + broker
PROCESSING	Worker toma el job, extrae y normaliza datos	PostgreSQL (update estado)
SUMMARIZING	LLM genera resumen/análisis del contenido	MongoDB (traza LLM) + metering
RENDERING	Se genera el documento de salida (JSON/PDF/DOCX)	Storage (S3/MinIO/filesystem)
COMPLETED	Documento listo para entrega	PostgreSQL (URL resultado) + webhook si aplica

Estado	Qué ocurre	Dónde se persiste
FAILED	Error en cualquier etapa; reintentos según política	PostgreSQL (error + retry_count) + MongoDB (traza)

18. Endpoints del Pipeline de Transformación

Endpoint	Método	Descripción	Respuesta
/v1/transform/submit	POST	Recibe JSON + parámetros de transformación	202 Accepted + job_id
/v1/transform/status/{job_id}	GET	Estado actual del job	200 con estado + progreso
/v1/transform/result/{job_id}	GET	Descarga el documento generado	200 con archivo o URL firmada
/v1/transform/cancel/{job_id}	POST	Cancela un job en cola o en proceso	200 si canceló, 409 si ya completó
/v1/transform/retry/{job_id}	POST	Reintenta un job fallido	202 Accepted + mismo job_id

19. Contratos de Entrada y Salida

19.1 TransformRequest (entrada)

```
{
  "tenant_id": "uuid",
  "user_id": "uuid",
  "source_integration_id": "uuid | null",
  "input_data": { ... },              // JSON libre (el dato a transformar)
  "transform_type": "summary | analysis | extraction | custom",
  "output_format": "json | pdf | docx",
  "priority": "normal | high",
  "callback_url": "https://... | null",  // Webhook para notificar al completar
  "metadata": { "tags": [], "department": "..." }
}
```

19.2 TransformResponse (salida de status)

```
{
  "job_id": "uuid",
  "status": "RECEIVED | QUEUED | PROCESSING | SUMMARIZING | RENDERING | COMPLETED |
FAILED",
  "progress_pct": 65,
  "created_at": "ISO8601",
```

```
  "updated_at": "ISO8601",
  "result_url": "https://... | null",   // Disponible cuando status=COMPLETED
  "error": { "code": "LLM_TIMEOUT", "message": "...", "retryable": true } | null
}
```

20. Capa de Rendering de Documentos

El output del LLM (texto resumido/estructurado) debe convertirse en un documento entregable. Esta capa se implementa como un módulo independiente dentro del backend:

Formato	Tecnología	Caso de uso
JSON	Serialización nativa (Pydantic)	Integraciones API-to-API
PDF	WeasyPrint o ReportLab	Documentos para usuarios finales
DOCX	python-docx	Documentos editables para usuarios

El documento generado se almacena en storage persistente (S3, MinIO o filesystem según infraestructura) con referencia en PostgreSQL (transform_jobs.result_url). TTL configurable para limpieza automática.

Parte VI — Capa de Integraciones

21. Adapter Pattern para Conectores

Cada integración externa se modela como un Adapter con un contrato común. Esto permite añadir nuevos conectores sin modificar el core del sistema.

```
class IntegrationAdapter(ABC):
    """Contrato común para todos los conectores"""

    @abstractmethod
    async def validate_credentials(self) -> bool: ...

    @abstractmethod
    async def receive_payload(self, raw: dict) -> NormalizedPayload: ...

    @abstractmethod
    async def deliver_result(self, job_id: str, result: TransformResult) ->
DeliveryReceipt: ...

    @abstractmethod
    async def health_check(self) -> HealthStatus: ...
```

Cada adapter normaliza los datos de entrada a un formato común (NormalizedPayload) antes de enviarlo al pipeline. Esto desacopla el formato específico de cada integración del procesamiento interno.

22. Registro de Integraciones en PostgreSQL

```
CREATE TABLE integrations (
    id uuid PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    connector_type text NOT NULL,           -- 'webhook', 'api_push', 'api_pull', etc.
    display_name text NOT NULL,
    credentials_ref text NOT NULL,          -- Referencia a vault, NUNCA plaintext
    config jsonb NOT NULL DEFAULT '{}',     -- Config específica del conector
    rate_limit_rpm int NOT NULL DEFAULT 60,
    is_active boolean NOT NULL DEFAULT true,
    health_status text NOT NULL DEFAULT 'unknown',
    last_sync_at timestamptz,
    last_error text,
    created_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE integrations ENABLE ROW LEVEL SECURITY;
CREATE POLICY integrations_isolation ON integrations
```

```
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

23. Colas de Trabajo con Prioridad por Tenant

La ingestión y transformación no pueden ser sincrónicas ni usar una cola global. Se implementa un sistema de colas con las siguientes características:

1. **Prioridad por tenant:** Cada tenant tiene fair-share de recursos. Un tenant que sube 10.000 documentos no bloquea a los demás.
2. **Dead letter queues:** Jobs que fallan repetidamente van a una DLQ para revisión manual sin bloquear la cola principal.
3. **Backpressure:** Si la cola supera un umbral, se rechazan nuevos jobs con 429 Too Many Requests hasta que baje.
4. **Circuit breaker por conector:** Si una integración falla repetidamente, se desactiva automáticamente y se alerta al administrador.
5. **Reintentos con backoff:** Reintentos exponenciales con jitter para evitar thundering herd.

Tecnología recomendada: Redis Streams para volumen moderado (< 10K jobs/hora). NATS JetStream o managed SQS/Pub/Sub si el volumen o la durabilidad lo requieren.

24. Circuit Breakers por Integración

Cada conector tiene un circuit breaker independiente que protege al sistema de integraciones defectuosas:

Estado	Comportamiento	Transición
CLOSED (normal)	Requests pasan al conector normalmente	Si fallan N consecutivas → OPEN
OPEN (cortado)	Requests se rechazan instantáneamente con error controlado	Después de timeout → HALF_OPEN
HALF_OPEN (prueba)	Se permite 1 request de prueba	Si éxito → CLOSED; si falla → OPEN

El estado del circuit breaker se persiste en Redis y se refleja en integrations.health_status en PostgreSQL para visibilidad en el dashboard de administración.

Parte VII — Metering y Costos por Tenant

25. Por Qué es Imprescindible

Sin metering no podés facturar, no podés aplicar quotas reales, y un tenant abusivo puede degradar el servicio de todos los demás. El metering es la base de la sostenibilidad económica del sistema multi-tenant.

26. Modelo de Datos

```
CREATE TABLE usage_metering (
    id bigserial PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    period_start date NOT NULL,              -- Inicio del período (día/mes)
    period_granularity text NOT NULL DEFAULT 'daily',
    embeddings_generated int NOT NULL DEFAULT 0,
    llm_calls int NOT NULL DEFAULT 0,
    llm_input_tokens bigint NOT NULL DEFAULT 0,
    llm_output_tokens bigint NOT NULL DEFAULT 0,
    documents_ingested int NOT NULL DEFAULT 0,
    documents_transformed int NOT NULL DEFAULT 0,
    storage_vectors_mb numeric NOT NULL DEFAULT 0,
    storage_mongo_mb numeric NOT NULL DEFAULT 0,
    api_calls_total int NOT NULL DEFAULT 0,
    UNIQUE(tenant_id, period_start, period_granularity)
);

ALTER TABLE usage_metering ENABLE ROW LEVEL SECURITY;
```

27. Qué se Mide

Recurso	Cómo se mide	Dónde se incrementa
Embeddings generados	Conteo por batch en ingestión	Paso 5 del Write Path
Llamadas LLM	Conteo por request (RAG + transformación)	Read Path paso 5 + Transform paso SUMMARIZING
Tokens LLM (input + output)	Metadata de la respuesta del LLM	Respuesta del proveedor de LLM
Documentos ingestados	Conteo por job completado	Paso 8 del Write Path
Documentos transformados	Conteo por job COMPLETED	Estado COMPLETED del Transform Pipeline
Storage vectorial (MB)	Cálculo periódico por colección/filtro	Job de background diario
Storage Mongo (MB)	db.stats() por tenant	Job de background diario

Recurso	Cómo se mide	Dónde se incrementa
API calls totales	Conteo por request en middleware	Middleware FastAPI

28. Quotas y Límites

Las quotas se definen por plan de tenant en PostgreSQL y se verifican en el middleware de FastAPI antes de procesar cualquier request costoso:

```
CREATE TABLE tenant_plans (
    tenant_id uuid PRIMARY KEY REFERENCES tenants(id),
    plan_name text NOT NULL DEFAULT 'free',
    max_documents_month int NOT NULL DEFAULT 100,
    max_llm_calls_month int NOT NULL DEFAULT 1000,
    max_transforms_month int NOT NULL DEFAULT 50,
    max_storage_mb int NOT NULL DEFAULT 500,
    rate_limit_rpm int NOT NULL DEFAULT 60
);
```

Si un tenant excede su quota, las operaciones costosas (ingestión, transformación, RAG) se rechazan con 429 Too Many Requests y un mensaje claro indicando qué límite se alcanzó y cuándo se renueva.

Parte VIII — Seguridad End-to-End

29. Principios de Seguridad

FERRO D2 implementa defensa en profundidad. Cada capa tiene controles propios; si una falla, las demás contienen el daño.

1. **Multi-tenant real:** Enforcement en DB (RLS) + validación en API + filtros en vector store + namespaces en cache.
2. **No exponer trazas:** Errores controlados y auditables; nunca stack traces al frontend ni a integraciones externas.
3. **Minimización de datos:** Guardar lo mínimo necesario en cada capa.
4. **Separación de instrucciones:** System prompt fijo + user prompt delimitado para mitigar prompt injection.
5. **Supply chain segura:** Dependencias escaneadas, imágenes firmadas, SBOM, secretos fuera del repo.
6. **Credenciales de integración:** Nunca en plaintext. Referencia a vault con rotación planificada. Cifrado en tránsito y reposo.

30. Modelo de Amenazas

Amenaza	Vector	Mitigación
Cross-tenant leakage	Error de filtrado en cualquier capa	tenant_id en cada tabla/colección/payload + RLS + tests automatizados
Prompt injection	Documentos con instrucciones maliciosas	Delimitación, política de herramientas, sanitización
Model output leakage	LLM filtra info entre sesiones/tenants	Aislamiento de memoria en Mongo por tenant+sesión, no compartir contexto entre tenants
Exfiltración por logs	Trazas contienen datos sensibles	Scrub + TTL + niveles de log; no loguear texto crudo
Poisoning de embeddings	Ingestas no confiables	Cuarentena, checksums, reindex controlado
Abuso de API	Tenant o integración abusiva	Rate limiting + quotas + circuit breakers por conector
SSRF en extractores	URLs maliciosas en documentos	Allowlist de URLs, sanitización de inputs

Amenaza	Vector	Mitigación
Integración comprometida	Credenciales de conector robadas	Rotación de credenciales, logs de acceso, circuit breaker

31. RBAC + ABAC + RLS

RBAC define roles base (viewer, editor, admin) por tenant. **ABAC** agrega políticas deterministas por atributos (sensibilidad, departamento, horario, nivel). **RLS** es el enforcement último en PostgreSQL. El vector store solo aplica filtros simples (tenant_id + doc_id).

32. Seguridad del Pipeline de Transformación

El pipeline bidireccional introduce superficies de ataque adicionales que requieren controles específicos:

1. **Validación del JSON de entrada:** Schema validation estricta. Rechazar payloads que excedan límites de tamaño o profundidad de anidamiento.
2. **Sanitización pre-LLM:** El contenido del JSON pasa por sanitización antes de enviarse al LLM para evitar prompt injection vía datos de integración.
3. **Aislamiento del output:** El documento generado se almacena en un namespace por tenant. URLs firmadas con expiración para descarga.
4. **Webhook seguro:** Los callback_url se validan contra allowlist de dominios. Se firma el payload de notificación con HMAC para que el receptor verifique autenticidad.
5. **Auditoría completa:** Input original, output generado, traza de LLM y decisión de policy se almacenan para cada job.

33. Seguridad del Bridge Rust (PyO3)

Airlock Pydantic fija límites de entrada. Rust los revalida (defensa en profundidad) y retorna errores controlados sin panics. Sin I/O en Rust reduce superficie de ataque. Fuzz testing recomendado para parsers de formatos complejos.

34. Seguridad Operativa

TLS end-to-end en todos los hops. Secretos en vault con rotación planificada. Contenedores non-root, filesystem read-only, seccomp/apparmor. Network segmentation: solo el gateway expone puertos públicos. Backups cifrados con restore drills.

Parte IX — Observabilidad y Operaciones

35. Observabilidad

Logs: JSON estructurado con correlation_id, tenant_id y job_id. Sin payload sensible por defecto.

Métricas: Latencia por endpoint, cache_hit ratio, tiempo de retrieval, tiempo de LLM, errores por dependencia, rate limit events, jobs en cola por tenant, tiempo promedio de transformación.

Trazas: Spans por etapa (authz, retrieval, LLM, rendering) con OpenTelemetry. Incluir job_id para correlacionar el pipeline completo de transformación.

Alertas: Picos de 401/403, timeouts LLM, errores Qdrant, crecimiento de colas, circuit breakers abiertos, tenants excediendo quotas, fallos de readiness.

36. Runbooks Operativos

Runbook	Procedimiento
Reindex Qdrant	Lock Redis → export IDs Postgres → purge Qdrant → re-embed batch → upsert → bump version → unlock
Rotación de secretos	Generar claves → actualizar vault → reiniciar servicios → verificar healthchecks
Incidente de fuga	Cortar tráfico → revocar tokens → auditar → notificar → remediar
Degradación de deps	Activar fallbacks → alertar → investigar
Integración caída	Circuit breaker abre → alertar admin → revisar credenciales/health → reactivar
DLQ overflow	Revisar jobs fallidos → clasificar errores → reprocessar o descartar → alertar tenant
Tenant excede quota	429 automático → notificar tenant → ofrecer upgrade o esperar renewal

37. Healthchecks Obligatorios

Endpoint	Tipo	Qué verifica
/healthz	Liveness	Proceso vivo (sin dependencias)
/readyz	Readiness	PostgreSQL (obligatorio) + Redis + Broker (opcionales según modo)
/version	Diagnóstico	Build, commit, versión, features habilitadas

Parte X — Guía de Implementación

38. Estructura del Monorepo

```
ferro-d2-root/
├── ops/
│   ├── docker-compose.yml
│   ├── docker-compose.prod.yml
│   ├── nginx/default.conf
│   ├── postgres/init.sql, rls.sql, metering.sql
│   ├── qdrant/config.yaml
│   └── scripts/  (backup, reindex, smoke_test, dlq_review)
├── apps/
│   ├── frontend/    (React + Vite + TanStack Query)
│   └── backend/
│       ├── pyproject.toml + Cargo.toml
│       ├── src/lib.rs                  (Rust / PyO3)
│       └── app/
│           ├── main.py                 (FastAPI)
│           ├── api/                    (health, chat, ingest, transform, integrations)
│           ├── core/                   (config, logging, security, metering)
│           ├── engine/                 (llamaindex_pipeline, rust_bridge)
│           ├── transform/              (pipeline, renderer, job_manager)
│           ├── integrations/           (base_adapter, connectors/, circuit_breaker)
│           ├── workers/                (ingest_worker, transform_worker)
│           ├── db/                     (postgres, redis, mongo, qdrant)
│           └── models/                 (auth, documents, chat, transform, integrations)
```

39. Motor Rust: Cargo.toml y lib.rs

```
# Cargo.toml
[package]
name = "ferro_core"
version = "0.1.0"
edition = "2021"

[lib]
name = "ferro_core"
crate-type = ["cdylib"]

[dependencies]
pyo3 = { version = "0.20", features = ["extension-module"] }
rayon = "1.8"
```

```
// src/lib.rs
use pyo3::prelude::*;
```

```
#[pyfunction]
fn rust_optimized_chunking(
    text: String, chunk_size: usize
) -> PyResult<Vec<String>> {
    let chunks = text.chars()
        .collect::<Vec<char>>()
        .chunks(chunk_size)
        .map(|c| c.iter().collect::<String>())
        .collect();
    Ok(chunks)
}

#[pymodule]
fn ferro_core(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(rust_optimized_chunking, m)?)?;
    Ok(())
}
```

40. Bridge LlamaIndex + Rust

```
# app/engine/rust_bridge.py
from llama_index.core.node_parser import NodeParser
from llama_index.core.schema import TextNode
import ferro_core  # Compiled Rust library

class RustNodeParser(NodeParser):
    chunk_size: int = 512

    def _parse_nodes(self, nodes, **kwargs):
        all_nodes = []
        for node in nodes:
            chunks = ferro_core.rust_optimized_chunking(
                node.get_content(), self.chunk_size
            )
            for chunk in chunks:
                all_nodes.append(
                    TextNode(text=chunk, metadata=node.metadata.copy())
                )
        return all_nodes
```

41. DDL Base: Multi-Tenant + RLS + Integraciones + Metering

```
-- Tenants y usuarios
CREATE TABLE tenants (
    id uuid PRIMARY KEY, name text NOT NULL,
```

```
    is_active boolean NOT NULL DEFAULT true,
    created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE users (
    id uuid PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    email text NOT NULL, full_name text NOT NULL,
    is_active boolean NOT NULL DEFAULT true,
    created_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY users_isolation ON users
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Integraciones
CREATE TABLE integrations (
    id uuid PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    connector_type text NOT NULL,
    display_name text NOT NULL,
    credentials_ref text NOT NULL,
    config jsonb NOT NULL DEFAULT '{}',
    rate_limit_rpm int NOT NULL DEFAULT 60,
    is_active boolean NOT NULL DEFAULT true,
    health_status text NOT NULL DEFAULT 'unknown',
    last_sync_at timestamptz,
    created_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE integrations ENABLE ROW LEVEL SECURITY;
CREATE POLICY integrations_isolation ON integrations
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Jobs de transformación
CREATE TABLE transform_jobs (
    id uuid PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    user_id uuid NOT NULL REFERENCES users(id),
    integration_id uuid REFERENCES integrations(id),
    status text NOT NULL DEFAULT 'RECEIVED',
    transform_type text NOT NULL,
    output_format text NOT NULL DEFAULT 'json',
    priority text NOT NULL DEFAULT 'normal',
    callback_url text,
    result_url text,
    error_code text, error_message text,
    retry_count int NOT NULL DEFAULT 0,
    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE transform_jobs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY jobs_isolation ON transform_jobs
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Metering
CREATE TABLE usage_metering (
    id bigserial PRIMARY KEY,
    tenant_id uuid NOT NULL REFERENCES tenants(id),
    period_start date NOT NULL,
    period_granularity text NOT NULL DEFAULT 'daily',
    embeddings_generated int DEFAULT 0,
    llm_calls int DEFAULT 0,
    llm_input_tokens bigint DEFAULT 0,
    llm_output_tokens bigint DEFAULT 0,
    documents_ingested int DEFAULT 0,
    documents_transformed int DEFAULT 0,
    storage_vectors_mb numeric DEFAULT 0,
    storage_mongo_mb numeric DEFAULT 0,
    api_calls_total int DEFAULT 0,
    UNIQUE(tenant_id, period_start, period_granularity)
);
ALTER TABLE usage_metering ENABLE ROW LEVEL SECURITY;

-- Plans y quotas
CREATE TABLE tenant_plans (
    tenant_id uuid PRIMARY KEY REFERENCES tenants(id),
    plan_name text NOT NULL DEFAULT 'free',
    max_documents_month int DEFAULT 100,
    max_llm_calls_month int DEFAULT 1000,
    max_transforms_month int DEFAULT 50,
    max_storage_mb int DEFAULT 500,
    rate_limit_rpm int DEFAULT 60
);
```

42. Docker Compose (Esqueleto con Broker)

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: ferro
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s; timeout: 3s; retries: 20

  redis:
```

```
    image: redis:7
    ports: ["6379:6379"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s; timeout: 2s; retries: 20

  mongo:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: [mongodata:/data/db]

  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333", "6334:6334"]
    volumes: [qdrantdata:/qdrant/storage]

  api:
    build: ../apps/backend
    environment:
      POSTGRES_DSN: postgresql://postgres:postgres@postgres:5432/ferro
      REDIS_URL: redis://redis:6379/0
      MONGO_URL: mongodb://mongo:27017/ferro
      QDRANT_URL: http://qdrant:6333
    ports: ["8000:8000"]
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }

  worker:
    build: ../apps/backend
    command: python -m app.workers.main
    environment: *api-env  # Same env as api
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }

volumes:
  pgdata:
  mongodata:
  qdrantdata:
```

Parte XI — Degradación Elegante y Fallbacks

Componente	Si falla...	Comportamiento
Redis	Cache no disponible	Mayor latencia, sin sesiones rápidas. Rate limit degradado.
MongoDB	Memoria no disponible	Responde sin historial. Trazas se encolan o descartan.
Qdrant	Vectores caídos	Respuesta sin contexto semántico o mensaje controlado.
PostgreSQL	Base de verdad caída	BLOQUEO TOTAL — Negar servicio antes que filtrar datos.
Message Broker	Cola caída	Jobs se rechazan con 503. Se alerta para resolución inmediata.
Integración externa	Conector falla	Circuit breaker abre. Jobs pendientes van a DLQ.
LLM Provider	API del LLM caída	Respuesta sin resumen/generación. Retry con backoff.

Principio inviolable: es preferible negar servicio completamente antes que filtrar datos o romper el aislamiento multi-tenant. PostgreSQL caído = sistema detenido.

Parte XII — Las Reglas de Hierro (Invariantes)

Estas reglas nunca deben violarse. Representan las decisiones de diseño más críticas:

1. **Soberanía de datos:** PostgreSQL es la fuente de verdad. Qdrant es un índice derivado reconstruible.
2. **Principio Airlock:** Ningún dato externo (incluyendo JSONs de integraciones) llega a Rust ni a las DBs sin validación Pydantic.
3. **Dopamine First:** Toda lectura verifica primero Redis. React aplica Optimistic UI.
4. **Vector store minimalista:** En Qdrant solo vectores y referencias compactas.
5. **Rust no hace I/O:** Solo cálculo CPU-bound puro. Sin red, sin DB, sin filesystem.
6. **Seguridad primero:** Si PostgreSQL cae, el sistema se detiene. Nunca filtrar datos.
7. **Errores controlados:** Nunca exponer stack traces. Ni al frontend, ni a integraciones externas.
8. **Aislamiento total:** tenant_id en CADA capa: Postgres, Mongo, Qdrant, Redis, colas, storage.
9. **Metering obligatorio:** Toda operación costosa se contabiliza. Sin metering no hay sostenibilidad.
10. **Circuit breaker por conector:** Una integración defectuosa no puede degradar el servicio global.

Parte XIII — Objetivos No Funcionales (NFRs)

Métrica	Objetivo	Notas
P95 API (sin RAG)	< 300ms	Endpoints CRUD y auth
P95 API (RAG + LLM)	< 1500ms	Depende del proveedor LLM
Cache hit Redis	< 5ms	First Hit Cache policy
Ingestión paralela	Rust + batch embeddings	Escala con cores
Transformación (end-to-end)	< 60s para docs estándar	Incluye LLM + rendering
Cola de jobs	Fair-share por tenant	Ningún tenant bloquea a otros
Degradación elegante	Sin stack traces	Fallback controlado por dependencia

Métrica	Objetivo	Notas
Trazabilidad	100% request/response LLM	Sin secretos en logs
Multi-tenant	Aislamiento completo	RLS + RBAC/ABAC + filtros en cada capa
Metering	Granularidad diaria mínimo	Base para facturación y quotas

Parte XIV — Checklist Go-Live

Verificar que todos los elementos estén implementados y probados antes de producción:

Área	Requisito	☐
API	Healthchecks /healthz y /readyz activos	
API	Auth + Rate limit + Pydantic + correlation_id	
API	Errores controlados (sin trazas al cliente ni a integraciones)	
API	Endpoints de transformación (submit/status/result) operativos	
Datos	PostgreSQL con RLS activo en TODAS las tablas multi-tenant	
Datos	Migraciones versionadas + auditoría	
Datos	Redis namespaces por tenant + expiración obligatoria	
Datos	MongoDB TTL en logs + scrub de secretos	
Datos	Qdrant payload mínimo + índices doc_id/tenant_id	
Integraciones	Adapter pattern implementado + circuit breakers	
Integraciones	Credenciales en vault (nunca plaintext)	
Integraciones	Webhooks con HMAC + allowlist de dominios	
Integraciones	Cola de jobs con prioridad + DLQ	
Metering	Contadores activos para embeddings, LLM, docs, transforms	
Metering	Quotas por plan + enforcement en middleware	
Seguridad	TLS end-to-end + headers seguros	
Seguridad	Secretos en vault + rotación planificada	
Seguridad	RBAC/ABAC testeado + RLS verificado	
Seguridad	Sanitización de prompts + políticas anti-injection	
Seguridad	Model output leakage: contexto aislado por tenant	
Ops	Docker multi-stage + CI con tests + scanning	
Ops	Backups cifrados + restore drills	
Ops	Observabilidad (métricas + logs + trazas con job_id)	
Ops	Runbooks documentados + fallbacks verificados	
Tests	Tests cross-tenant automatizados	
Tests	Integration tests con compose completo	
Tests	Tests de transformación end-to-end (submit → COMPLETED)	

Área	Requisito	☐
Tests	Tests de circuit breaker y DLQ	

Anexo A — AI System Skills (Prompt Operativo)

Copiar y pegar en la configuración de cualquier agente de IA para adoptar el rol de Arquitecto FERRO D2 v4:

```
Eres el Arquitecto Principal del stack FERRO D2 v4 (LlamaIndex Edition).

Directivas:
1) Stack estricto: FastAPI, LlamaIndex (v0.10+), React, Rust (PyO3), Docker.

2) D2 Router:
   - Redis: cache/sesiones/rate limit (< 5ms) + job status + locks.
   - Postgres: verdad (ACID), tenants/usuarios/roles/permisos/integraciones/metering/jobs.
   - Mongo: logs y memoria (chat history + trazas LLM + transform payloads).
   - Qdrant: embeddings + payload mínimo (doc_id/tenant_id/chunk_id).

3) Reglas de hierro:
   - Rust no hace I/O: solo CPU-bound.
   - Nada entra a Rust sin Pydantic (Airlock).
   - Nunca chunking/tokenización en Python puro.
   - Postgres decide permisos; Qdrant se consulta con doc_ids permitidos.
   - Fallback seguro: si Qdrant falla, responder sin contexto y sin trazas.
   - tenant_id en CADA capa: PG, Mongo, Qdrant, Redis, colas, storage.
   - Toda operación costosa se contabiliza en metering.

4) Integraciones bidireccionales:
   - Adapter pattern para conectores con contrato común.
   - Cola de jobs asíncronos con prioridad por tenant.
   - Circuit breaker por conector.
   - Pipeline de transformación: RECEIVED > QUEUED > PROCESSING >
     SUMMARIZING > RENDERING > COMPLETED.

5) Dopamine UX:
   - First-hit cache en Redis.
   - Frontend con Optimistic UI y polling/WebSocket para jobs.

© 2026 Zeron Architecture Team.
```

—— *Fin del Documento* ——

© 2026 Zeron Architecture Team / Antigravity — Confidencial